

DL Midsem Notes — Section 1

Linear Algebra and Basic Maths for DL

Not directly PYQ-tested, but every later derivation (backprop, CNN params, PCA-flavoured intuition) leans on this. Skim fast if it's already second nature; the point is fluency, not depth.

1 Theory

1.1 Vectors and Dot Products

A vector $\mathbf{x} \in \mathbb{R}^n$ is a list of numbers. Dot product: $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$. This is exactly what a neuron's weighted sum $\sum w_i x_i$ is — a dot product between the weight vector and the input vector. When $\mathbf{a} \cdot \mathbf{b} = 0$ the vectors are orthogonal.

1.2 Norms

$$\|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

L_1 measures "total magnitude", L_2 measures Euclidean length. These reappear directly as L1/L2 regularization penalties on weight vectors (section 9).

1.3 Matrices

A layer's weights form a matrix $W \in \mathbb{R}^{k \times n}$ (k outputs, n inputs); the forward pass for a whole layer is $\mathbf{a} = W\mathbf{x} + \mathbf{b}$, one matrix-vector product replacing k separate dot products. Matrix multiplication $(AB)_{ij} = \sum_k A_{ik} B_{kj}$ is *not* commutative in general. Transpose W^T swaps rows/columns; used constantly in backprop when pushing a gradient back through a layer ($\delta_{\text{in}} = W^T \delta_{\text{out}}$).

1.4 Eigenvalues and Eigenvectors

For a square matrix A , $A\mathbf{v} = \lambda\mathbf{v}$: $\mathbf{v}$ is a direction A only stretches (by factor λ), never rotates. Why this matters for DL: the Hessian (matrix of second derivatives) of a loss surface has eigenvalues that tell you the curvature along each direction — large eigenvalue = sharp curvature (needs a small step), small eigenvalue = flat direction (needs a large step). This is the root cause of why a single global learning rate is awkward (motivates Adam/RMSProp in section 7).

1.5 Basic Calculus for DL

Derivative $f'(x)$: instantaneous rate of change. Partial derivative $\partial f / \partial x_i$: rate of change along one axis, holding others fixed. Gradient $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$: the vector of all partials, points in the direction of steepest *increase*. Chain rule: $\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$ — the entire justification for backpropagation is repeated application of this one rule through a deep composition of functions.

2 Practice Questions

Q1. Given $\mathbf{a} = (3, 4)$, $\mathbf{b} = (1, 2)$, compute $\mathbf{a} \cdot \mathbf{b}$, $\|\mathbf{a}\|_2$, and $\|\mathbf{a}\|_1$.

Q2. Multiply $A = \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$, $B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$: compute AB and confirm $AB \neq BA$.

Q3. Find the eigenvalues and eigenvectors of $A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$.

Q4. For $f(x, y) = x^2y + 3y^2$, compute ∇f at the point $(1, 2)$.

Q5 (concept check). In one or two lines: why does the gradient point in the direction of steepest *ascent*, and how does that fact justify the negative sign in gradient descent?

3 Answers

A1. $\mathbf{a} \cdot \mathbf{b} = 3(1) + 4(2) = 11$. $\|\mathbf{a}\|_2 = \sqrt{9 + 16} = 5$. $\|\mathbf{a}\|_1 = 3 + 4 = 7$.

A2. $AB = \begin{pmatrix} 1(0) + 2(1) & 1(1) + 2(0) \\ 3(0) + 1(1) & 3(1) + 1(0) \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$. $BA = \begin{pmatrix} 0(1) + 1(3) & 0(2) + 1(1) \\ 1(1) + 0(3) & 1(2) + 0(1) \end{pmatrix} = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$. $AB \neq BA$, confirming non-commutativity.

A3. A is diagonal, so eigenvalues are the diagonal entries directly: $\lambda_1 = 2$, $\lambda_2 = 3$, with eigenvectors $\mathbf{v}_1 = (1, 0)$, $\mathbf{v}_2 = (0, 1)$ (the standard basis vectors, since A scales each axis independently).

A4. $\partial f / \partial x = 2xy$, $\partial f / \partial y = x^2 + 6y$. At $(1, 2)$: $\partial f / \partial x = 2(1)(2) = 4$, $\partial f / \partial y = 1 + 12 = 13$. So $\nabla f(1, 2) = (4, 13)$.

A5. By Cauchy–Schwarz, the directional derivative $\nabla f \cdot \hat{\mathbf{d}}$ is maximized (most positive) when $\hat{\mathbf{d}}$ points exactly along ∇f — that’s the direction of fastest increase. To *decrease* f fastest we go the opposite way, $-\nabla f$; that’s precisely the update $w \leftarrow w - \eta \nabla f$.